

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

*I **THAMIZHARASAN S (92424087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

THAMIZHARASAN S

Annexure A

Short Answer Test

1. Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.

ANSWER:

Comparison of K-Nearest Neighbour and Case-Based Learning

Consider a medical diagnosis system that predicts whether a patient has a particular disease based on symptoms, age, blood pressure, test results, etc.

Feature	K-Nearest Neighbour (KNN)	Case-Based Learning (CBL)
Basic idea	Finds the K most similar patients and uses their labels to predict the new patient's disease.	Finds a previously solved medical case that is similar and adapts its solution to the new patient.
Memory usage	Stores most or all training data, so memory usage can be high.	Stores previous cases, including their descriptions, solutions, and sometimes explanations. Can require more structured memory.
Similarity measure	Usually uses mathematical measures such as Euclidean distance, Manhattan distance, or cosine similarity.	Uses a case similarity function designed for the problem. Different medical features can have different importance.
Prediction strategy	Uses the majority class among the K nearest patients. Example: 4 out of 5 similar patients have diabetes → predict diabetes.	Retrieves the most relevant previous case(s), reuses/adapts their diagnosis or treatment, and may store the new solved case.

Feature	K-Nearest Neighbour (KNN)	Case-Based Learning (CBL)
Basic idea	Finds the K most similar patients and uses their labels to predict the new patient's disease.	Finds a previously solved medical case that is similar and adapts its solution to the new patient.
Learning	Usually called a lazy-learning method because little training occurs beforehand.	Learns through case retrieval, reuse, revision, and retention.
Adaptation	New data can be added directly to the stored dataset.	New medical cases can become new cases in the case library after being solved.

Medical Example

Suppose a new patient has:

- Age = 50
- Blood pressure = High
- Glucose = High
- BMI = 31

Using KNN:

The system calculates the distance between this patient and existing patients. It selects the nearest K patients. If most of them were diagnosed with diabetes, the system predicts diabetes.

Using Case-Based Learning:

The system searches its case library for a patient with a similar combination of age, symptoms, glucose, blood pressure, and BMI. If a highly similar previous case was diagnosed with diabetes, the system uses that case as a basis and adapts the previous diagnosis to the new patient.

Conclusion

KNN is simpler and more mathematical: store data → calculate similarity → select K neighbours → vote. Case-Based Learning is more flexible and reasoning-oriented: retrieve a similar past case → reuse/adapt its solution → revise if necessary → store the new case.

For medical diagnosis, KNN is useful when numerical patient data is dominant, while Case-Based Learning is better when previous cases contain rich information, explanations, and expert solutions.

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

ANSWER:

Both **Locally Weighted Regression (LWR)** and **Radial Basis Function (RBF) networks** are used for **function approximation**, but they handle locality and computation differently.

Example Dataset

Consider a dataset representing the relationship between **temperature (°C)** and **ice-cream sales**:

Temperature (x)	Sales (y)
10	20
15	30
20	45
25	65
30	90
35	120
40	150

Suppose we want to predict sales when the temperature is **27°C**.

Locally Weighted Regression (LWR)

LWR gives **higher weights to data points closer to 27°C** and lower weights to points farther away.

For example:

- 25°C → high weight
- 30°C → high weight
- 15°C → low weight
- 40°C → very low weight

It fits a small regression model around the query point and produces the prediction.

Main characteristics:

- **Locality:** Very high. It focuses on nearby training points for every prediction.
- **Computational cost:** High during prediction because weights and a local regression must be calculated for each new input.
- **Smoothness:** Depends strongly on the bandwidth. A small bandwidth can produce more flexible predictions; a large bandwidth produces smoother predictions.
- **Memory:** Usually stores the training data because it needs the data during prediction.

Radial Basis Function (RBF) Network

An RBF network uses hidden neurons centered at selected points in the input space. Each neuron responds strongly when the input is close to its center.

For example, centers might be selected around:

15°C, 25°C, 35°C

For an input of 27°C, the neuron centered at 25°C will have a stronger response than the neurons centered at 15°C or 35°C.

The outputs of these neurons are combined to produce the final sales prediction.

Comparison

Feature	LWR	RBF Network
Locality	Uses nearby data points directly for each prediction	Uses fixed radial basis functions around learned centers
Computation	High at prediction time	More computation during training, usually faster prediction
Training	Very little/no global training	Requires learning centers, widths and output weights
Smoothness	Controlled mainly by bandwidth	Controlled by basis-function widths
Memory	Can require storing the whole dataset	Usually stores only learned parameters/centers
Best suited for	Small datasets and highly local relationships	Larger datasets and repeated predictions

Key Difference

The biggest difference is when the computation happens:

- **LWR:** *“Calculate a local model whenever I need a prediction.”*
- **RBF:** *“Learn a set of basis functions first, then use them to make predictions.”*

Therefore, **LWR is more locally adaptive but computationally expensive at prediction time**, while **RBF networks require more training but generally provide faster prediction and smooth function approximation**.

3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.

ANSWER:

K-Nearest Neighbour vs. Radial Basis Function in Classification

Consider a medical classification problem where patients are classified as Diabetic or Non-Diabetic using features such as glucose level and BMI.

Example

Suppose the training data contains:

- Low glucose + low BMI → **Non-Diabetic**
- High glucose + high BMI → **Diabetic**
- A new patient has glucose = 145 mg/dL and BMI = 29.

K-Nearest Neighbour (KNN)

KNN finds the K closest patients to the new patient using a distance measure such as Euclidean distance.

If (K=5), and 4 of the 5 nearest patients are diabetic, KNN predicts:

Diabetic

KNN does not create a fixed model during training. It mainly stores the training data and performs the calculations when a new sample arrives.

Radial Basis Function (RBF) Network

An RBF network first learns **centres** from the training data. Each hidden neuron measures how close the new patient is to its centre using a radial basis function.

The outputs of the hidden neurons are then combined to classify the patient as diabetic or non-diabetic.

Comparison

Feature	KNN	RBF Network
Decision boundary	Can be highly irregular because it depends directly on nearby training samples.	Usually smoother because boundaries are formed from overlapping radial basis functions.
Training time	Very low; mainly stores training data.	Higher because centres, widths and output weights must be learned.
Prediction time	Slower, especially with large datasets, because distances to many training points must be calculated.	Generally faster after training because only the learned network is evaluated.
Generalization	Can work well locally but may overfit when K is too small.	Often provides better smooth generalization when centres and widths are chosen properly.
Memory	Can require storing the entire training dataset.	Usually stores a smaller set of learned parameters.
Flexibility	Very flexible and adapts directly to new training examples.	Less directly dependent on individual examples but captures broader patterns.

Decision Boundary

KNN:

If the diabetic and non-diabetic patients are mixed together, KNN can create a complex boundary that bends around individual data points.

RBF:

The radial functions create overlapping smooth regions, so the resulting boundary is generally smoother.

Conclusion

KNN is better when the dataset is small and local similarity is important. It has almost no training cost but can become slow for large datasets.

RBF networks are better when repeated predictions and smooth generalization are required. They take more time to train, but after training they usually predict faster and can produce smoother decision boundaries.

4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.

ANSWER:

Case-Based Learning vs. Locally Weighted Regression

Consider a **movie recommendation system** that recommends movies based on a user's age, rating history, preferred genre, and previous viewing behaviour.

Case-Based Learning (CBL)

Case-Based Learning stores previous user cases and their solutions. When a new user arrives, the system searches for users with similar preferences and recommends movies that worked well for them.

For example, if a new user likes action and science-fiction movies, the system retrieves similar users and recommends movies they enjoyed.

Locally Weighted Regression (LWR)

LWR uses nearby data points to build a local regression model. For example, it can predict the rating a user might give to a movie based on ratings from users with similar characteristics.

Nearby users receive higher weights, while dissimilar users receive lower weights.

Comparison

Feature	Case-Based Learning	Locally Weighted Regression
Handling new data	Retrieves similar previous cases and reuses/adapts their solutions.	Calculates weights for nearby data and builds a local model for each prediction.

Adaptability	Highly adaptable because new solved cases can be added to the case library.	Adaptable because predictions are based on nearby data and change as new data is added.
Computational efficiency	Can be efficient if the case database is indexed well, but searching many cases can be expensive.	Usually computationally expensive at prediction time because distances, weights, and local regression must be calculated.
Type of prediction	Reuses a previous solution or recommendation and adapts it.	Produces a numerical prediction using a local regression model.
Memory requirement	Stores previous cases, including their attributes and solutions.	Usually needs access to the training dataset during prediction.
Best use	Recommendation based on similar past users/items.	Predicting a continuous value such as expected movie rating.

Example

Suppose a new user has:

- Age: 21
- Likes: Action, Sci-Fi
- Previous rating: 4.5/5

CBL:

Finds previous users with similar preferences and recommends movies they liked.

LWR:

Finds nearby users based on similarity, gives the most similar users higher weights, and predicts the new user's rating for a particular movie.

Conclusion

Case-Based Learning is better for recommendation systems where past experiences and solutions are important. It can directly reuse successful recommendations and adapt them to new users.

Locally Weighted Regression is better when the goal is to predict a numerical value, such as a user's expected rating. However, it can be computationally expensive because a local model must be calculated for each new prediction.

5. Compare Decision Tree Learning and K-Nearest Neighbour (KNN) using a classification problem such as customer churn prediction. Analyze differences in model interpretability, training time, handling of noisy data, and decision-making process. Discuss how each method performs with large datasets and complex feature spaces.

ANSWER:

Consider a customer churn prediction system for a telecom company. The goal is to predict whether a customer will leave the company (Churn) or stay (No Churn) using features such as monthly bill, contract type, customer age, tenure, and number of complaints.

Example

Suppose a customer has:

- Age = 25
- Tenure = 6 months
- Monthly bill = ₹1,500
- Complaints = 4
- Contract = Monthly

The system predicts whether this customer is likely to **churn**.

Decision Tree Learning

A Decision Tree makes predictions by following a sequence of if-then decisions.

For example:

Contract = Monthly?

→ Yes → Complaints > 3?

→ Yes → Predict: Churn

The tree is trained beforehand by selecting useful features and splitting the dataset into different branches.

K-Nearest Neighbour

KNN does not build a decision tree. For a new customer, it calculates the distance between that customer and existing customers.

For example, with **K = 5**, if 4 of the 5 most similar customers churned, the prediction is:**Churn**

Comparison

Feature	Decision Tree	KNN
Interpretability	Very high. Decisions can be explained using simple rules.	Lower. Prediction depends on similarity to neighbouring customers.
Training time	Usually higher because the tree must be constructed.	Very low because it mainly stores the training data.
Prediction time	Usually fast; follows a path from root to leaf.	Can be slow because distances to many training samples may need to be calculated.
Noisy data	Can overfit noise if the tree becomes too deep; pruning helps.	Sensitive to noisy or irrelevant features because they affect distance calculations.
Decision-making	Uses learned rules and feature splits.	Uses the majority class of nearby data points.
Large datasets	Generally handles large datasets better, especially with appropriate tree algorithms.	Prediction becomes expensive as the dataset grows.

Complex feature spaces	Handles nonlinear relationships and mixed feature types well.	Can struggle when there are many irrelevant features because of the curse of dimensionality .
Feature scaling	Usually not required.	Usually required when features have different scales.

Handling Noisy Data

Decision Tree:

A very deep tree can memorize noisy training examples, causing overfitting. This can be reduced using pruning, maximum depth, or minimum samples per leaf.

KNN:

Noise directly affects the nearest-neighbour calculation. A noisy or irrelevant feature can make two customers appear more or less similar than they actually are. Choosing a suitable **K** and scaling/selecting features can reduce this problem.

Large and Complex Datasets

For large customer datasets, Decision Trees are generally more practical because they can learn a compact set of rules and make predictions quickly.

KNN becomes expensive because every new customer may need distance calculations against many stored customers. It also becomes less effective when there are hundreds of features, because meaningful neighbours become harder to identify.

Conclusion

Decision Tree is generally the better choice for customer churn prediction when interpretability, scalability, and fast prediction are important. Managers can easily understand rules such as “Monthly contract + many complaints → high churn risk.”

KNN is useful when similar customers tend to have similar outcomes, especially with smaller datasets and carefully selected features. However, its prediction cost and sensitivity to high-dimensional or noisy data make it less suitable for very large customer databases.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

